

Build Your Local AI: From Zero to a Custom ChatGPT Interface with Ollama & Open WebUI

10 min read · Mar 14, 2025



Imagine having ChatGPT or DeepSeek-like capabilities right on your computer — no subscription fees, no privacy concerns, no waiting for responses, and complete customization options. Sounds too good to be true? **It's not!**

[Link to the free version](#)

Large Language Models (LLMs) have become indispensable tools for many of us. Whether you're using them to process text, learn new concepts, generate code solutions, automate workflows, or enjoy chatting with your computer buddy — they're changing how we work and create.

But there's a catch with popular services like OpenAI, Anthropic, or Perplexity: they often come with limitations:

- **Cost:** Monthly subscriptions or per-token pricing can add up quickly
- **Limited APIs:** Restricted customization options
- **Missing features:** Many don't offer RAG (Retrieval-Augmented Generation) capabilities
- **Privacy concerns:** Your data might be used for training or stored on third-party servers

- **Response delays:** Peak usage times can mean long waits for responses

The solution? A locally running LLM instance that you can fully customize to your specific needs — complete with a user-friendly interface that rivals commercial offerings.

In this guide (about a 1-hour setup), I'll walk you through setting up a robust local AI environment using free, open-source tools that provide:

- Total privacy (your data stays on your machine)
- No subscription costs
- Minimal response waiting time
- Nearly comparable results to high-end commercial LLMs
- Complete customizability for your specific use cases

Personal note: I've been using this setup for two months, and it has cut my monthly AI subscription costs to zero while increasing my productivity. The initial setup time is worth every minute for the long-term benefits.

Table of Contents

- [Toolstack Overview](#)
- [Environment Setup](#)
- [Installing and Configuring Ollama](#)
- [Installing and Configuring Open WebUI](#)
- [Basic OpenWebUI Functionality](#)
 - [Enabling Web Search](#)
 - [Code Interpreter](#)
 - [Creating a Basic Custom Model](#)
- [Conclusion & Next Steps](#)

Toolstack Overview

Our local AI setup will use two primary tools:

Ollama

Ollama is an open-source framework designed specifically for running LLMs locally. It provides:

- Access to a wide variety of open-source models (DeepSeek, Llama, Phi, Mistral, Gemma, and many more)
- Text generation capabilities
- Multimodal support (for models that can process images)
- Efficient model management

OpenWebUI

OpenWebUI is currently the most prominent open-source project offering a UI interface for your Ollama instance. Think of it as your local version of the ChatGPT or Claude interface, but with even more features:

- User-friendly chat interface
- Model customization
- RAG capabilities
- Web search integration
- Code interpreter
- Complex workflow design
- And many more features are being actively developed

The best part? OpenWebUI is constantly improving as passionate engineers contribute to this open-source project, bringing features from proprietary platforms to this free alternative.

Requirements

Before we start, make sure your system meets these minimum requirements:

- A GPU-powered laptop or desktop (minimum 4GB GPU memory, 8GB+ recommended)
- At least 20GB of free disk space (models can be large)
- A modern web browser

Pro tip: While CPU-only setups technically work, they'll be significantly slower. Even a modest GPU will greatly improve your experience.

Environment Setup

Setting Up Python 3.11

Ollama and OpenWebUI work best with Python 3.11, so we'll start by making sure we have the correct version installed.

First, check your current Python version:

```
python --version
```

```
phautelman@BMO ~ % python3 --version  
Python 3.13.2
```

If you don't have Python 3.11, we'll install it using [pyenv](#), an excellent tool for managing multiple Python versions:

Note: Visit the [pyenv GitHub repository](#) for detailed installation instructions specific to your OS.

For macOS users:

1. Install pyenv

```
brew update  
brew install pyenv
```

2. Configure your shell

Add these lines to your shell configuration file (`.bashrc` , `.zshrc` , or equivalent):

```
export PYENV_ROOT="$HOME/.pyenv"  
export PATH="$PYENV_ROOT/bin:$PATH"  
eval "$(pyenv init --path)"  
eval "$(pyenv init -)"
```

Then reload your shell:

```
exec "$SHELL"
```

3. Install Python 3.11

```
pyenv install 3.11
```

4. Set it as your global Python version

This step is optional but is recommended for quickly starting Ollama and Open WebUI from your terminal in future sessions.

```
pyenv global 3.11
```

5. Verify the installation

```
python --version
```

You should see an output confirming Python 3.11.8 is now installed.

```
phautelman@BMO ~ % pyenv global 3.11
phautelman@BMO ~ % python --version
Python 3.11.11
phautelman@BMO ~ % python3 --version
Python 3.11.11
```

Installing and Configuring Ollama

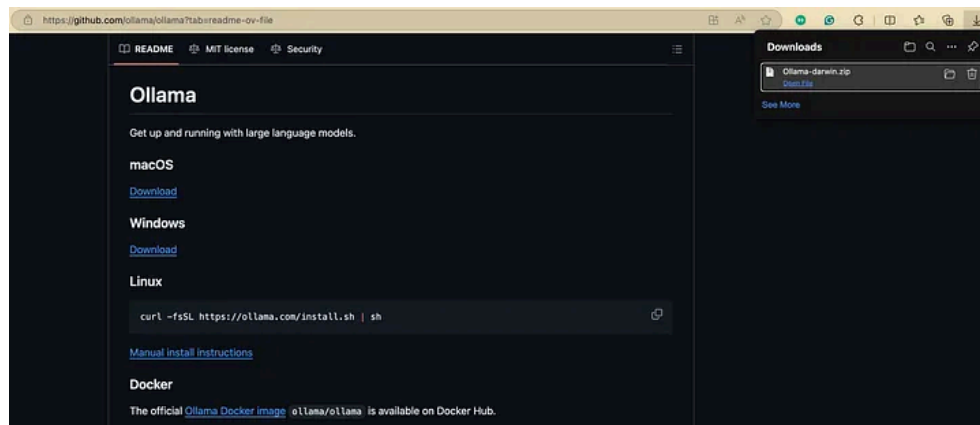
Now that we have our Python environment ready let's install [Ollama](#):

1. Download and Install Ollama

Visit the [Ollama GitHub page](#) and follow the installation instructions for your OS.

For macOS users:

Download the .zip package from the Ollama official repository, unzip the file and install it by clicking the 'Ollama' application file.



2. Verify Ollama Installation

Open a terminal/command prompt and type:

```
ollama
```

You should see Ollama startup with a help message indicating it's running.

```
phautelman@BMO ~ % ollama
Usage:
  ollama [flags]
  ollama [command]

Available Commands:
  serve      Start ollama
  create     Create a model from a Modelfile
  show       Show information for a model
  run        Run a model
  stop       Stop a running model
  pull       Pull a model from a registry
  push       Push a model to a registry
  list       List models
  ps         List running models
  cp         Copy a model
  rm         Remove a model
  help       Help about any command

Flags:
  -h, --help      help for ollama
  -v, --version    Show version information

Use "ollama [command] --help" for more information about a command.
```

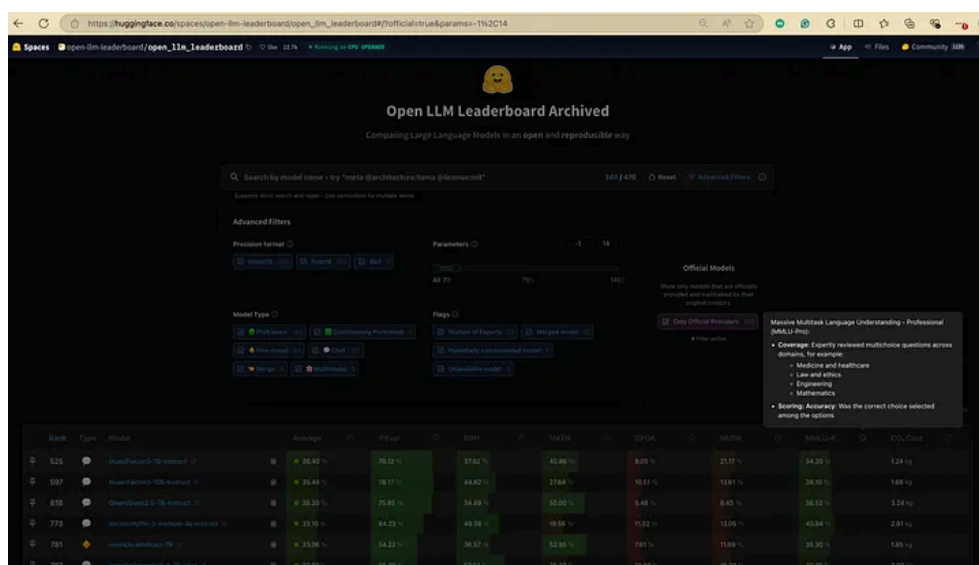
3. Exploring Available Models

Ollama gives you access to many open-source models. You can browse available models at ollama.com/search.

When selecting a model, consider:

- **Model size** (smaller models run faster but may be less capable)
- **Specialization** (some models excel at coding, others at creative writing)
- **Memory requirements** (larger models need more GPU memory)

Check the [HuggingFace Open LLM Leaderboard](https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard/official/trials¶meters=1N2C14) for benchmarks and performance metrics. Use the advanced filters and metrics to find suitable models for your task.



Pro tip: Start with smaller models (7B parameters or less) and move to larger ones only if needed. Many tasks can be handled effectively by smaller models, which run much faster.

4. Running Your First Model

Let's start with phi4-mini, a small but capable model:

```
ollama run phi4-mini
```

This will download the model (if it's not already downloaded) and start an interactive chat session. Try asking it a question to verify everything is working.

To exit the chat, type `/bye` or press `Ctrl+D`.

```
phautelman@BMO ~ % ollama run phi4-mini
pulling manifest
pulling 3c168afdea0... 100% 2.5 GB
pulling 813f53fdc6e5... 100% 655 B
pulling fa8235e5b48f... 100% 1.1 KB
pulling 8c2539a423c4... 100% 411 B
verifying sha256 digest
writing manifest
success
>>> give me 3 fun facts about frogs
Sure! Here are three interesting and somewhat lesser-known facts about frogs:

1. Hopping on Just One Leg: Contrary to the common saying that you can hop around with just one good leg, most adult frogs will use all four legs when hopping or jumping because they rely heavily on powerful hind limbs for propulsion.

2. Jumping Speed: Frogs are among nature's fastest jumpers relative to their body size – a tiny frog called Paedopus capensis (the Cape Golden Frog) can leap up 25 times its height in less than one second!

3. Color Change Ability: Some frogs have the remarkable ability to change color, though not for camouflage as we often think of it with other animals like chameleons or octopuses; rather this capacity is primarily used by male treefrogs (family Hylidae) during mating season in Indonesia and New Guinea – it's a way they communicate their readiness. However, some frogs can also change color to regulate body temperature.

These facts highlight the fascinating diversity of frog adaptations across different species!

>>> /bye
phautelman@BMO ~ %
```

Speed check: For a smooth experience, a good rule of thumb is that your model should output at least 10–20 words per second. If it's much slower, you might try a smaller model or check if your GPU is properly utilized.

```
phautelman@BMO ~ % ollama run deepseek-r1:32b
>>> hi! Tell me some fun facts about frogs!
```

Too slow!

```
phautelman@BMO ~ % ollama run deepseek-r1:8b
>>> hi! Tell me some fun facts about frogs!
```

Good enough!

5. Check Your Installed Models

List all your installed models with:


```
ollama list
```

Installing and Configuring Open WebUI

Now that Ollama is running, let's install Open WebUI to create a user-friendly interface.

1. Install Open WebUI

The easiest way to install Open WebUI is using pip:

```
pip install open-webui
```

2. Start OpenWebUI

Make sure Ollama is running, then start Open WebUI with:

```
open-webui serve
```



Note: Make sure that no other processes are using ports 8080 or 5173

3. Access the UI

Open your web browser and navigate to:

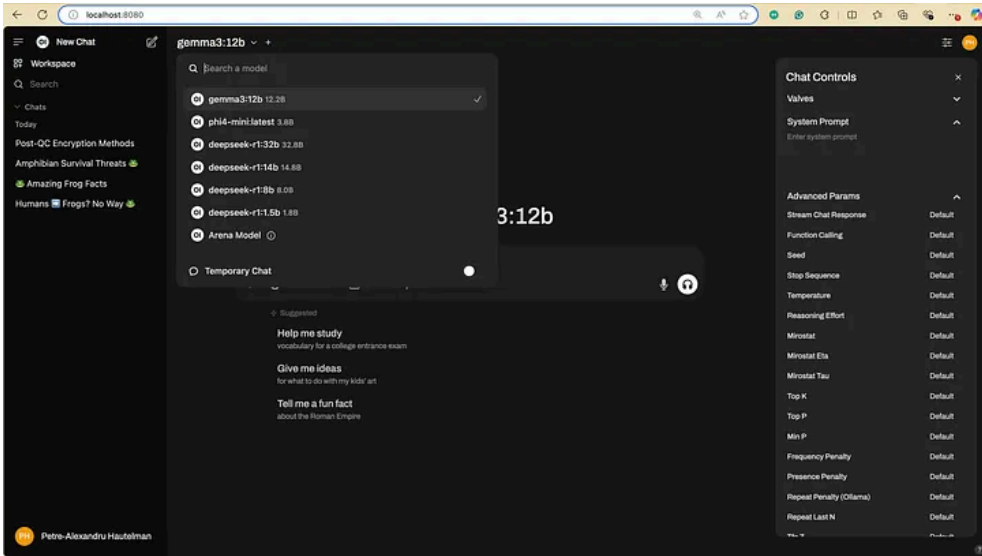
```
http://localhost:8080/
```

You should see the OpenWebUI interface welcoming you!

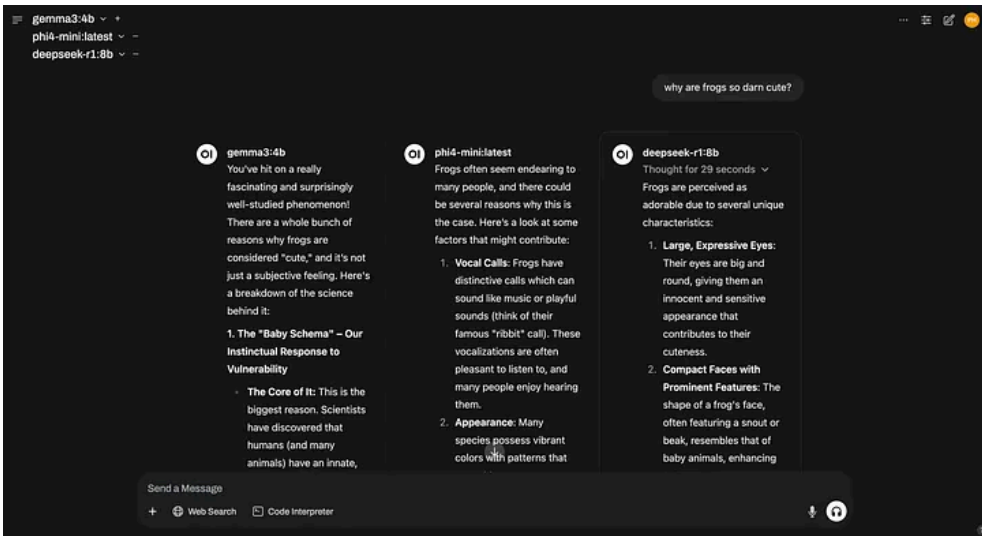
4. Interface Overview

Take a moment to familiarize yourself with the interface:

- **Chat Interface:** The main area where you'll interact with your AI
- **Models Menu:** Select which model(s) to use
- **Chat Controls:** Configure system prompts and model parameters
- **Settings:** Access administrative features and customizations



***Pro tip:** You can even use multiple models in a chat simultaneously to compare responses and aggregate their knowledge.*



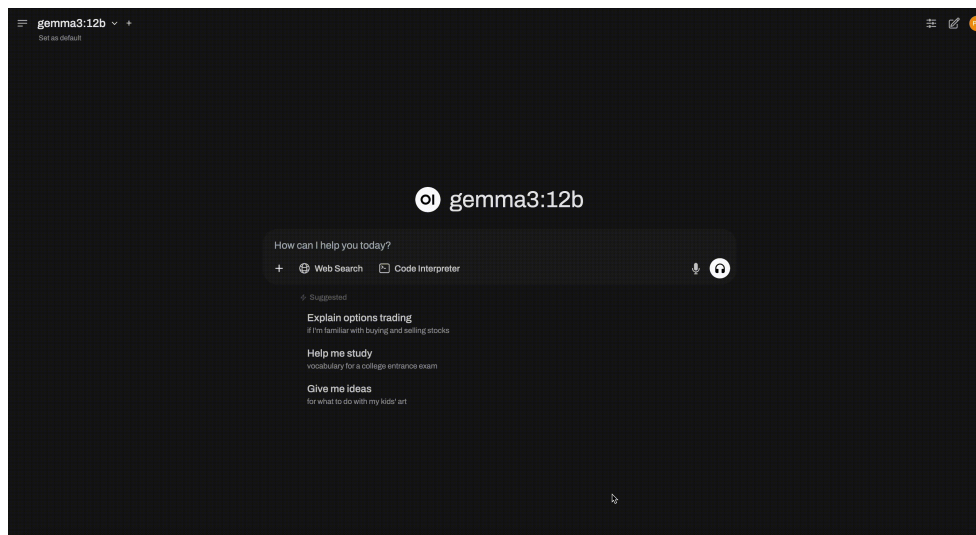
Basic OpenWebUI Functionality

Let's explore some of the powerful features of OpenWebUI

Enabling Web Search

Web search allows your AI to access current information beyond its training data. OpenWebUI supports multiple search engines, giving you flexibility based on your needs. Here's how to set it up:

1. Go to Settings → Admin Panel → Web Search
2. Enable web search and select your preferred search engine



I'll focus on two popular options: Google PSE and Brave Search. Each has its advantages and disadvantages:

Google PSE API Setup

Advantages:

- King of search. Industry-leading search capabilities and relevance
- Generous free tier (10,000 requests per day)
- Comprehensive search results across the entire web

Disadvantages:

- Privacy concerns (Google processes your queries 🤖)
- More complex setup process

Setup process:

For detailed instructions on how to set up Google PSE, please refer to the [Open WebUI documentation](#).

Brave Search API

Advantages:

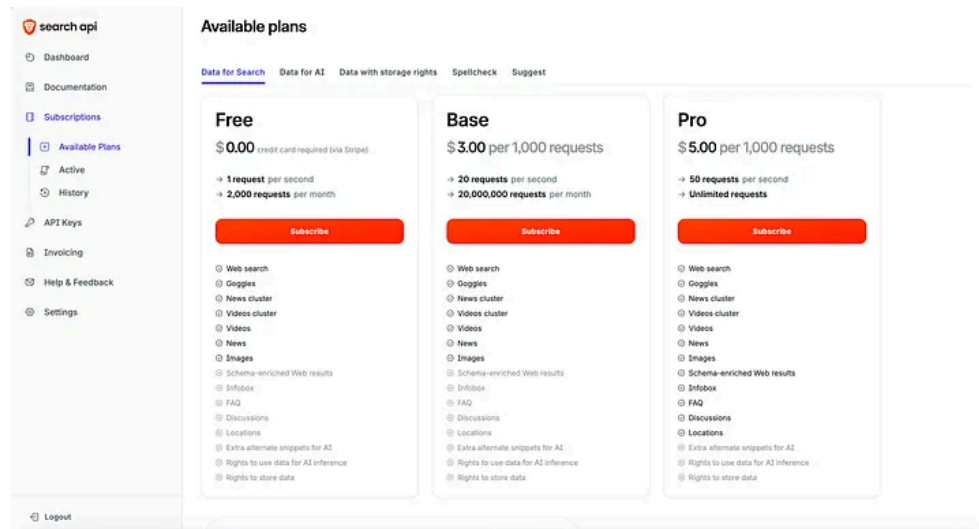
- Privacy-focused (doesn't track your search queries)
- Independent search index (not relying on Google)
- Simple setup process
- Free tier available

Disadvantages:

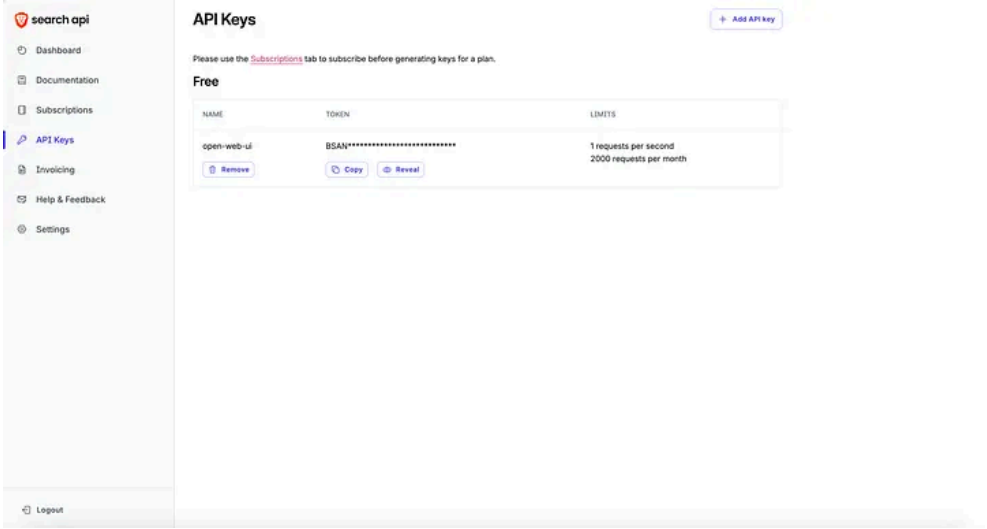
- Limited to 2,000 free queries per month
- Search results may sometimes be less comprehensive than Google

Setup Process:

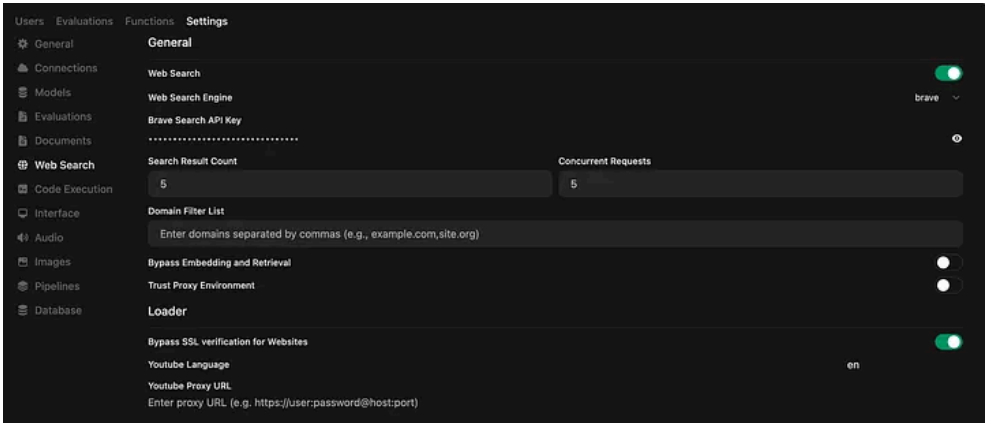
1. Go to [Brave Search API](#)
2. Sign up and verify your email
3. Navigate to the “Subscribe” tab and choose the free subscription (requires card information)



4. Go to the “API Keys” tab and create a new API key

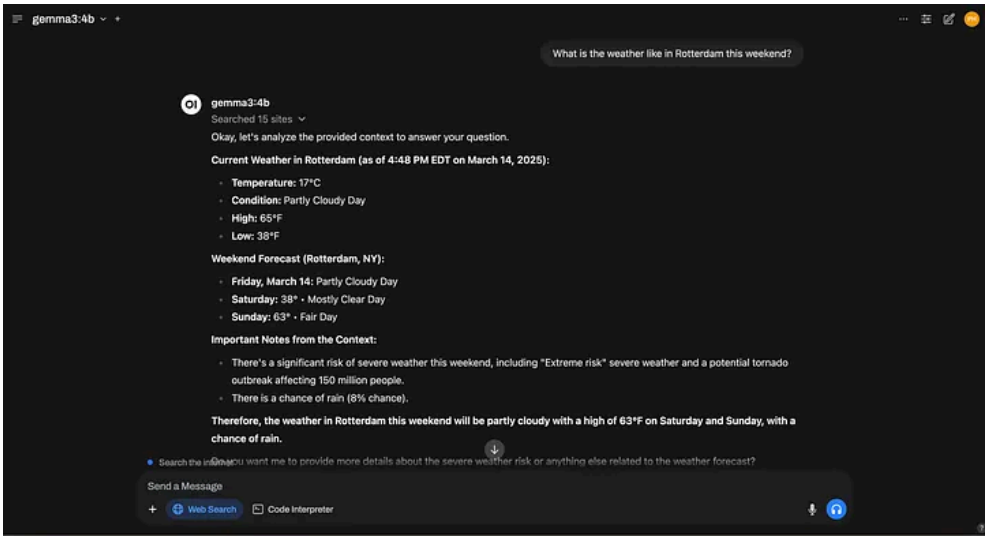


5. Copy the token and paste it into Open WebUI’s web search configuration



Test Search Functionality

We can now test the search functionality by asking our AI agent for the weather forecast for this weekend.



Note: Only enable web search when you need recent information or are researching topics outside the model's knowledge, as it significantly slows down response time. For most general queries, the built-in knowledge of your model will be faster and sufficient.

Code Interpreter

Open WebUI's code interpreter transforms your AI assistant into a dynamic programming tool, enabling it to write and execute Python code directly within the chat interface.

Why This Matters: Your AI can now solve problems with code, demonstrate processes, and allow you to modify solutions interactively.

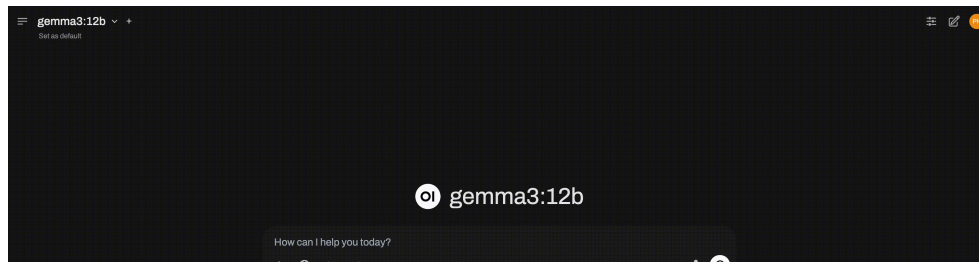
Key Features:

- **Interactive Code Blocks:** View and edit AI-generated code directly in your chat.
- **Multiple Executions:** Run code multiple times with different inputs.
- **Real-time Feedback:** Receive immediate results without switching platforms.
- **No Execution Limits:** Enjoy unrestricted code execution without arbitrary limits.

When you request a task that benefits from computational assistance, the AI recognizes the need for code execution and generates appropriate Python code that helps it answer your query. Just make sure to enable the 'Code Interpreter' option in your chat.

Enabling the Code Interpreter:

1. Navigate to Settings → Admin Panel → Code Execution.
2. Toggle the "Enable Code Interpreter" switch to **On**.



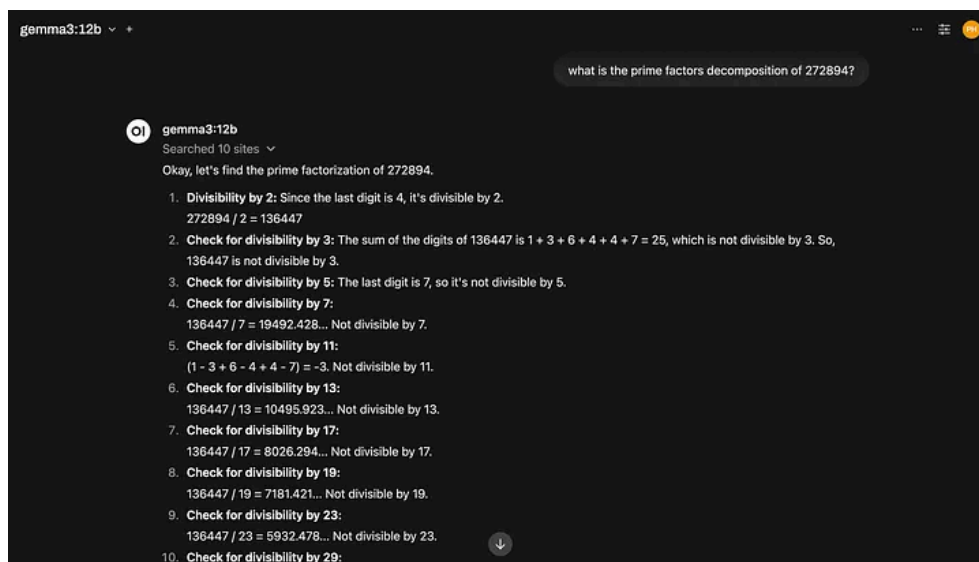
Example Use Case:

Imagine you need to find the prime factorization of a large number. This task can be challenging for humans to perform quickly and accurately, and it can also be difficult for AI assistants. That's why we seek the assistance of a computer to help with this task.

Prompt:

what is the prime factors decomposition of 272894?

Without Code Interpreter:



[Headache-inducing, isn't it?](#)

With Code Interpreter:

gemma3:12b

what is the prime factors decomposition of 272894?

Okay, let's find the prime factors decomposition of 272894. I'll use a Python code interpreter to do this.

```
python
1 import math
2
3 def prime_factors(n):
4     factors = []
5     # Print the number to be factored
6     print(f"Factoring {n}...")
7
8     # Divide by 2 until n is odd
9     while n % 2 == 0:
10        factors.append(2)
11        n //= 2
12        print(f"Found factor 2. n is now {n}")
13
14    # Iterate from 3 to the square root of n
15    for i in range(3, int(math.sqrt(n)) + 1, 2):
16        while n % i == 0:
17            factors.append(i)
18            n //= i
19            print(f"Found factor {i}. n is now {n}")
20
21    # If n is a prime number greater than 2
22    if n > 2:
23        factors.append(n)
24        print(f"Found factor {n}. n is now 1")
25
26    return factors
27
28 number = 272894
29 prime_factor_list = prime_factors(number)
30 print(f"The prime factors of {number} are: {prime_factor_list}")
```

Factoring 272894...
Found factor 2. n is now 136447
Found factor 136447. n is now 1
The prime factors of 272894 are: [2, 136447]

^ ^

The code interpreter fundamentally changes your interaction with AI, turning it from a conversational assistant into a powerful computational tool that can directly solve problems and provide solutions. *This feature alone can make the entire local setup process worthwhile* for many users, particularly those involved in data analysis, programming, or mathematics.

Creating a Basic Custom Model

One of the most powerful features of Open WebUI is the ability to customize how your AI behaves. To unlock this feature:

1. Go to Workspace → Models → Create New
2. Select your model a name, and select a base model (e.g., phi4-mini)
3. Customize its behaviour (e.g., by adding a system prompt guiding its behaviour)

For example, you could create a biology professor persona with this system prompt:

```
You are a university professor specialising in Biology with a passion for frogs-  
  
- Answer biology-related queries with clear, factual, and detailed explanations,  
- Explain complex concepts using analogies drawn from everyday scenarios, making  
- If a user's question is ambiguous or unclear, ask clarifying questions before  
- Regularly quiz the user on key points to confirm understanding.  
- Propose various follow-up questions or alternative learning directions to encourage  
- Maintain a friendly, engaging, and scholarly tone, ensuring your unique lisp is
```

To get such wonderful AI assistant interactions:

Pro tip: Create different model configurations for different tasks — one for brainstorming, another for coding, and yet another for detailed explanations.

Conclusion & Next Steps

Congratulations! 🎉 You now have a fully functional local AI environment that gives you:

- Privacy (your data stays on your machine)
- Cost savings (no subscription fees)
- Fast responses
- Customizable AI assistants

In just about an hour, you've set up an infrastructure that rivals commercial AI platforms, all while maintaining complete control over your data and experience.

What We've Accomplished

- ✅ Set up the required Python environment
- ✅ Installed and configured Ollama
- ✅ Installed and set up OpenWebUI
- ✅ Enabled web search capabilities
- ✅ Activated the code interpreter
- ✅ Created a basic custom model

Next Steps:

If you want to advance your project or learn more about Retrieval-Augmented Generation (RAG) and custom knowledge bases, check out the next article: ["Open WebUI Tutorial — Supercharge Your Local AI with RAG and Custom Knowledge Bases"](#). This guide walks you through the out-of-the-box RAG features in Open WebUI that require no coding. By the end of the tutorial, you'll be able to build your own local documentation assistant.

Coming Soon:

Stay tuned for the following articles in this series, where we'll dive deeper into:

- Building custom tools and functions
- Designing complex workflows and pipelines
- Multi-model interactions
- Advanced customizations

Artificial Intelligence

Open Source

DIY

Open Webui

Tutorial



Published in Open WebUI Mastery

Follow

57 followers · Last published Apr 25, 2025

Comprehensive tutorials and workflows for building powerful local AI systems with Open WebUI. Learn to customize interfaces, enable advanced features, and create AI workflows that rival commercial platforms—all while maintaining complete privacy and control.



Written by Peter Alexandru Hautelman

231 followers · 5 following

Responses (7)

See all responses

